Freight Load Rate Prediction

Approach, validation design and results

1. Objective and data

The goal is to predict posted_rate (USD) for freight loads. The labeled development data (train-test.csv) has 48,000 loads from Jan 1 to Oct 31, 2025. The 12,000 loads to predict (validation.csv) run from Nov 1 to Dec 31, 2025. A separate 31-row file supplies the December chart inputs.

2. Train / validation split

Every load to be predicted is later in time than every labeled load, so the task is forecasting, not interpolation. A random split would mix future and past rows and flatter the results. I used a time-based holdout instead:

- **Training period:** Jan 1 to Aug 31 (38,477 rows).
- **Holdout period:** Sep 1 to Oct 31 (9,523 rows), the most recent two months, which mimics predicting Nov and Dec.
- **Metrics:** MAE, RMSE, median absolute error and MAPE in dollars, reported on all holdout rows and on clean rows only (see section 3), because a small set of corrupted labels dominates the all-rows error.
- **Final model:** refit on all clean development rows (Jan to Oct) before predicting the validation and December files.

3. Data-quality issues and fixes

Issue	How it was handled
624 of 48,000 training rows have an implausible rate per mile (below $0.90 or above $5)	Excluded from training. Metrics shown both with and without them.
292 negative weights (sign flips)	Converted to absolute value and flagged.
300 missing weights	Filled with the median and flagged.
374 training and 249 validation rows missing market_index	Filled with that date's average, since it barely varies within a day.
Reported distance inconsistent with city coordinates (187 training, 39 validation rows)	Replaced with 1.18 x straight-line distance and flagged. Distances at the 70-mile floor are kept as reported; an early version of this rule wrongly altered them and produced near-zero predictions, which I caught in a sanity check and fixed.
December chart rows lack coordinates, market_index and quote_signal	Coordinates from a city lookup; market_index from the date average in validation.csv; quote_signal from the monthly average.

4. Models and results

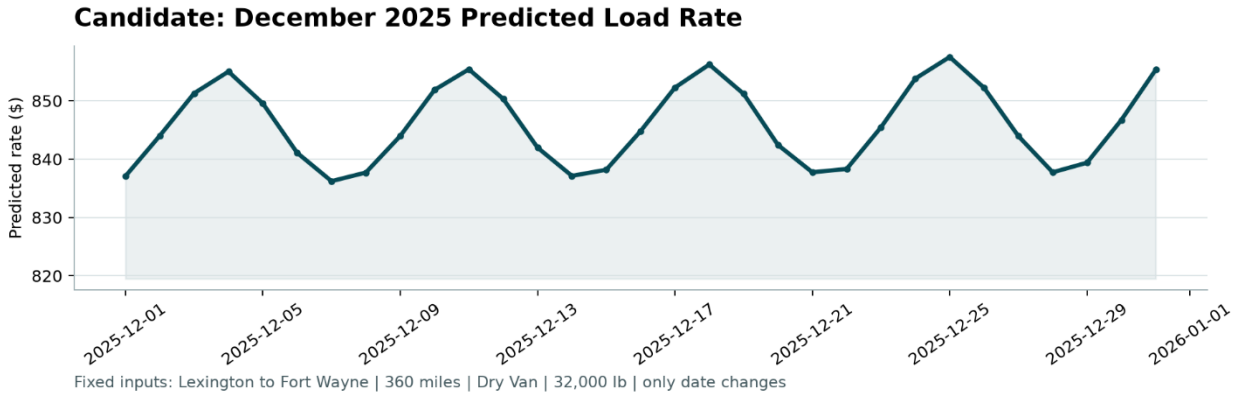
Features: log and straight-line distance, equipment, weight, market index, quote signal, pickup and delivery city, coordinates and their differences, and calendar features. The target is log(posted_rate), converted back to dollars for scoring. Four models were compared on the holdout:

Model	MAE (all)	MAE (clean)	RMSE (clean)	MAPE (clean)
Ridge (log target)	105.29	53.21	159.49	2.31%
Random Forest	112.24	60.15	164.33	2.50%
LightGBM	109.04	57.05	163.09	2.38%
Hybrid: Ridge + LightGBM on residuals (final)	**104.66**	**52.60**	**159.38**	**2.25%**

Price is largely linear in log distance with city and equipment effects, so the regularized linear model beats both tree ensembles on its own. The hybrid keeps Ridge as the backbone and fits a small, heavily regularized LightGBM to its residuals, which gives the best MAE, clean-row RMSE and MAPE. The gain over Ridge is small, so I chose the hybrid mainly on its consistent edge rather than a large margin. A Ridge alpha sweep from 0.3 to 30 moved MAE by only about 0.2, so the model is not sensitive to that setting.

5. December prediction chart

The chart below was produced by the provided score.py from validation_predictions.csv and data/december_chart_inputs.csv.



Predicted rates for the Lexington to Fort Wayne route sit between about $836 and $858 across December. Day-to-day movement is small because the chart rows have no real quote_signal, so the main driver of variation is the market index.

6. Limitations

- **Corrupted labels:** they are about 1.3% of training rows but roughly double holdout MAE (about 105 vs 53). No model can predict them, so the clean-row metrics reflect typical accuracy.
- **December inputs:** a monthly-average quote_signal removes real daily variation from the chart.

- **Unseen routes:** 736 validation routes never appear in training. The model leans on distance, equipment and city-level effects rather than memorizing routes, which is why route-level encoding was not used.